Information Security - TP: Breaking the Classics

Malek
Universite Mohammed VI Polytechnique (UM6P)
College of Computing

October 2026

Introduction

This practical work is an escape-room exercise in classical cryptography. The goal is to transform and break messages encrypted with a Caesar cipher, a monoalphabetic substitution cipher, and a Vigenere cipher. Each stage gives a clue for the next one. The supplied TP sheet defines the questions, and `Caesar_cipher.py` contains the student's current Python work.

The method followed here is *inspect the given ciphertext, test the cipher rule, verify the plaintext, and explain the result*. Figures show code captures made directly from the current Python file. They preserve its unfinished parts rather than presenting corrected code as if it were the original submission.

1 Working Files and Method

The TP handout provides the messages, a fixed substitution alphabet, and hints for recovering a four-letter Vigenere key. The Python file contains Caesar routines, a frequency-based substitution attempt, and the final Vigenere solver. The reported exercise answers were checked independently against the handout; the program was also run to record its actual output.

2 Caesar Cipher Warm-Up

2.1 Encryption and Decryption

A Caesar cipher shifts each letter by a fixed key k. With A=0 through Z=25, encryption is C=(P+k) mod 26 and decryption is P=(C-k) mod 26. Spaces are left unchanged. The code capture below shows the attempted encryption and decryption functions in the source file.



Figure 1: Caesar encryption and decryption functions in the supplied Python file.

Using k=4, `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` becomes `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. Decrypting the handout's k=8 ciphertext gives `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. The unusual first word is retained exactly as supplied; it is not silently edited to a contraction.

Observation. These two outputs are independently verified. The displayed `encypt` and `decypt` functions do not currently produce both outputs: `aphabet` is undefined, and the decryption branch is indented under the wrong loop level.

2.2 Exhaustive Search

A Caesar cipher has only 26 possible keys. Trying all of them is simpler than estimating a key statistically. The existing brute-force function loops over keys 0 to 25 and prints candidate plaintexts.

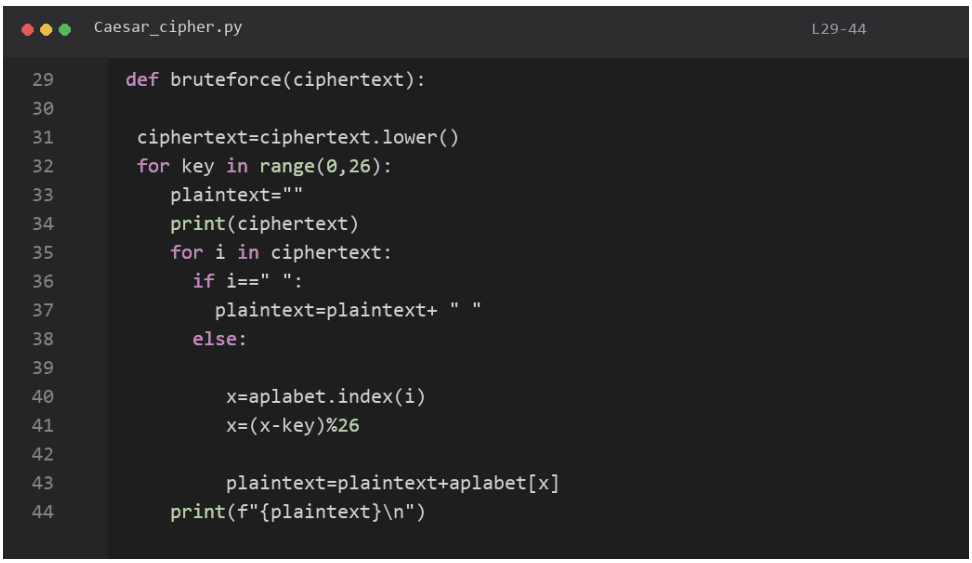


Figure 2: The 26-key Caesar brute-force loop.

For `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP`, the coherent candidate occurs at k=11: `THE KEY IS UNKNOWN TRY BRUTE FORCE`. The source currently repeats the ciphertext for every key; printing each key alongside its candidate would make the result easier to inspect.

3 The Letter Wall: Substitution Cipher

3.1 Known Substitution Alphabet

In a monoalphabetic substitution, each plaintext letter is replaced by one fixed ciphertext letter. The TP gives a complete mapping for its first two questions, and the same two alphabets are defined in the Python file.

Caesar_cipher.py L50-57

```
50  plain  = ['A','B','C','D','E','F','G','H','I','J','K','L','M',
51            'N','O','P','Q','R','S','T','U','V','W','X','Y','Z']
52
53  cipher = ['Q','M','J','Z','T','G','F','K','P','W','L','S','B',
54            'O','X','N','C','R','Y','E','V','H','I','A','D','U']
55  def decrypt(text):
56   clean=""
57   feq="ETROSAHNDIGLUYMVCPWFKBJXQZ"
```

Figure 3: Plain and cipher alphabets defined in the source file.

Applying this mapping gives `KEEP THE SECRET SAFE` to `LTTN EKT YTJRTE YQGT`. The reverse mapping gives `ERVYE DXVR SXFPJ` to `TRUST YOUR LOGIC`. The file stores the alphabets, but its running code does not yet perform these known-key operations.

3.2 Unknown Alphabet and Frequency Analysis

When the alphabet is unknown, there are 26! possible permutations, approximately 4.03 x 10^26. Brute force is not practical. The intercepted paragraph contains 197 letters after removing spaces and punctuation. The most frequent ciphertext letter is T (34 times, 17.3%), followed by E (19, 9.6%) and R (18, 9.1%). Frequency gives a first guess, while repeated words and context refine it.

Caesar_cipher.py L60-79

```
60      for k in range (0,26):
61        for i in text:
62           if i==plain[k]:
63              frq[i]=frq[i]+1
64      result = sorted(frq.items(), key=lambda item: item[1], reverse=True)
65      al=''
66      for i in result:
67         al=al+i[0]
68      print(al)
69
70
71      for p in text:
72             if p in al:
73                x=al.index(p)
74                clean=clean+feq[x]
75
76             else:
77                clean=clean+p
78
79      print(clean)
```

Figure 4: Frequency ranking and provisional letter replacement in the Python file.

The recovered message says that the secret message is hidden in the paragraph, letter frequency gives a good start but not the whole key, and the Vigenere key has four letters. The current program produces an approximation beginning `THE SECRET UESSAGE...`; this is evidence that raw frequency ranking alone needs contextual correction. It also does not calculate the percentages requested in the TP.

4 Breaking the Vigenere Cipher

4.1 Four Caesar-Like Groups

A four-letter key repeats across positions 0,1,2,3, then again at 4,5,6,7. Separating positions by their remainder modulo four creates four groups, each encrypted with one Caesar shift. The code assumes that E is the most frequent plaintext letter in each group, as suggested by the handout.

```python
from collections import Counter


def break_vigenere(ciphertext, key_length=4):
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    text = "".join(character for character in ciphertext.upper() if character in alphabet)
    shifts = []

    for position in range(key_length):
        group = text[position::key_length]
        most_common_letter = Counter(group).most_common(1)[0][0]
        shifts.append((alphabet.index(most_common_letter) - alphabet.index("E")) % 26)

    key = "".join(alphabet[shift] for shift in shifts)
    plaintext = "".join(
        alphabet[(alphabet.index(character) - shifts[position % key_length]) % 26]
        for position, character in enumerate(text)
    )
    return key, plaintext
```

Figure 5: Final Vigenere solver: four groups, shift estimates, and decryption.

The leading ciphertext letters of the four groups are G, S, H, and I. Comparing each with E gives shifts 2, 14, 3, and 4, corresponding to the key letters C, O, D, and E. Thus the recovered key is `CODE`. The key is verified by decrypting the entire 198-letter ciphertext into coherent English rather than relying on frequency alone.

4.2 Input and Observed Result

```python
vigenere_ciphertext = (
    "VVHWGQUIVYHIRSUPGTWXJSVIPCWIUPHGCIVIVVHPGHWITSDTRSDVUSYITMZL"
    "GFHIXSUCVSDQOSPFGFVLQIOHEVHGMSDGJQRPWAQWGSNXJSPSUHIVGEXIPHVC"
    "OPRPCBGYUSLXVCUIECYITHKIMSBAJSQXJSPIUGDKGPHGQAHWEZHETSQXGFWL"
    "GOFGGGVAQFGIEZLTUS"
)

vigenere_key, vigenere_plaintext = break_vigenere(vigenere_ciphertext)
print("Vigenere key:", vigenere_key)
print("Vigenere plaintext:", vigenere_plaintext)
print("Access word:", vigenere_plaintext.split("ACCESSWORD")[-1])
```

Figure 6: Vigenere ciphertext and printed result statements in the current code.

The program output confirms the key `CODE`. The final sentence of the decrypted message instructs the reader to enter the access word `ECLIPSE`. The screenshot below records the terminal result from running the supplied Python file.

```
PS> python Caesar_cipher.py
Vigenere key: CODE
Vigenere plaintext: THESECRETKEEPERLEFTTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTE
    AMMEMBERSHOULDCHECKEACHCOLUMNSEEKTHEMOSTFREQUENTSYMBOLANDUSEITTORECOVERTHEKEYWHENTHEME
    SSAGEBECOMESCLEARENTERTHEACCESSWORDECLIPSE
Access word: ECLIPSE
```

Figure 7: Terminal output: recovered key, plaintext, and access word.

5 Verification and Limitations

The Caesar exercise answers and known substitution results above were checked independently from the TP sheet. The unknown substitution text was checked against consistent repeated-letter mappings. The Vigenere key was checked by decrypting all 198 ciphertext letters. The current Python script still has faults in the earlier Caesar functions and only a partial substitution attack, so its successful Vigenere output should not be read as proof that every previous function works.

The frequency assumption in each Vigenere group is a heuristic. It succeeds here because the groups are long enough and the resulting plaintext is meaningful. For a shorter or unusual message, one should test several candidate shifts instead of automatically treating the top letter as E.

6 Conclusion

The TP demonstrates why classical ciphers fail in different ways. Caesar has a tiny key space and can be exhausted. Substitution has many keys but exposes language structure. Vigenere distributes the shifts until its repeated key positions are separated. The final verified key is `CODE` and the final access word is `ECLIPSE`. The captured code and terminal result show both the completed final stage and the corrections still needed in the earlier parts.

Sources: Imane Fouad, *TP: Breaking the Classics: A Cryptography Escape Room*, UM6P; supplied `Caesar_cipher.py`, inspected and run for this report.